Les bases de Tkinter

Créer une interface graphique avec Tkinter et Python 3

Par Vincent Maillol

Date de publication : 26 juin 2016

Dernière mise à jour : 26 juin 2016

Cet article montre comment créer des IHM pour Python 3 avec la boite à outils graphique de base, Tkinter.

Commentez

I - Introduction....3
II - Concepts de base....3
III - Les widgets disponibles dans Tkinter....5
III-A - Le Label....6
III-B - Les Boutons (Button)....6
III-C - Les champs de saisie (Entry)....7
III-D - Les widgets checkbutton....8
III-E - Les widgets Radiobutton....9
III-F - La Spinbox....10
III-G - Le widget Listbox....11
III-H - Le widget Scale....12
IV - Les conteneurs....12
IV-A - Les fenêtres (Toplevel)....13
IV-B - le widget Frame....14
IV-C - LabelFrame....15
IV-D - le widget PanedWindow....15
V - Un peu plus loin avec grid()....15
VI - Les événements....17
VII - Organiser son code....19
VII-A - Le widget Canvas....20
VII-A-1 - Créer des lignes....20
VII-A-2 - Créer des rectangles....21
VII-A-3 - Créer des ovales....22
VII-A-4 - Créer des arcs....22
VII-A-5 - Créer des polygones....23
VII-A-6 - Interagir avec les objets du Canvas....24
VII-A-7 - Avoir des événements sur les objets du Canvas....25
VII-A-8 - Appliquer des tags sur les objets du Canvas....26
VII-B - Les animations....26
VII-C - Le widget Scrollbar....27
VIII - Conclusion....28
IX - Remerciements....28

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

I - Introduction

Tkinter est une boite à outils pour faire des interfaces graphiques en Python. Elle repose sur un autre langage de script, le Tcl (*Tool Command Language*) en utilisant la bibliothèque Tk de ce langage. Tkinter est l'abréviation de « Tk interface ».
Tkinter s'installe sous Windows lors de l'installation de **Python**. Sous Linux, on peut vérifier son installation avec la commande :

```
1. python3 -m tkinter
```

Si vous avez un message d'erreur qui s'affiche, vous pouvez installer Tkinter avec la ligne de commande suivante.

Pour un système basé sur Debian :

```
1. apt-get install python3-tk
```

Pour un système basé sur Red Hat :

```
1. yum install python3-tkinter
```

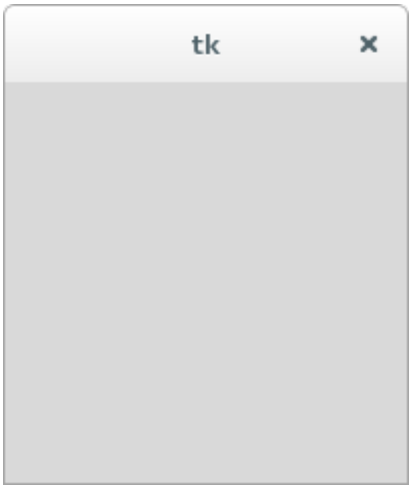
II - Concepts de base

Le fonctionnement est le suivant : on dispose de plusieurs éléments graphiques appelés « widgets », certains widgets sont destinés à en contenir d'autres, c'est notamment le cas des fenêtres. La règle étant que tout widget doit être contenu dans un autre, excepté un widget spécial, qui est la fenêtre racine.

Pour créer une interface graphique avec Tkinter, il y a deux choses à faire : créer une fenêtre racine et lancer la boucle principale via la méthode **mainloop()**. L'appel à cette méthode bloque l'exécution de l'appelant. Tkinter est alors à l'écoute des événements provenant de l'utilisateur, tels que des clics de souris.

```
1. from tkinter import Tk
2. root = Tk() # Création de la fenêtre racine
3. root.mainloop() # Lancement de la boucle principale
```

Une fenêtre avec le titre « tk » s'affiche.



Lorsqu'on ajoute des widgets, il faut d'abord créer le widget, puis le positionner en utilisant un gestionnaire de position.

Tous les widgets prennent en premier paramètre le widget dans lequel ils seront contenus.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

Les widgets ont en commun de nombreux attributs, que l'on pourra paramétrer comme ceci lors de leur construction :

```
1. un_widget = UnWidget(widget_parent, un_parametre='une_valeur')
```

On peut également paramétrer un widget après sa création en utilisant la méthode **configure()** ou son alias **config()** :

```
1. un_widget.config(un_parametre='une_valeur')
```

Pour récupérer la valeur d'un paramètre d'un widget, on utilisera la méthode **cget**() :

```
1. un_widget.cget('un_parametre')
```

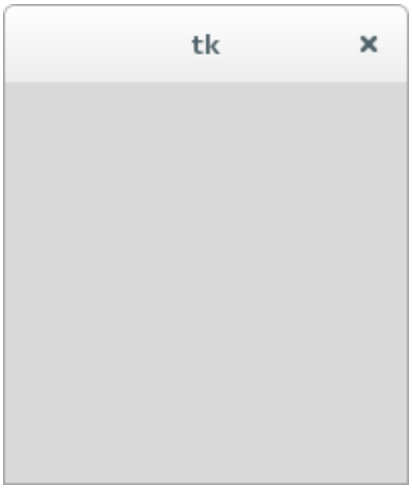
Une autre façon pour lire ou changer l'attribut d'un widget est d'utiliser **__getitem__** :

```
1. un_widget['un_parametre'] # accéder à un attribut
2. un_widget['un_parametre'] = 'une_valeur' # Modifier un attribut.
```

Ainsi pour créer un Label avec le texte « Hello », on va faire ceci :

```
1. from tkinter import Tk, Label
2. root = Tk() # Création de la fenêtre racine
3. label = Label(root, text='Hello')
4. root.mainloop() # Lancement de la boucle principale
```

Lorsqu'on exécute notre code, notre label ne s'affiche pas, comme indiqué au début de ce chapitre, il faut positionner notre label après l'avoir créé.



Pour positionner un widget, Tkinter propose trois gestionnaires de position. Chacun permet de placer les widgets avec une philosophie différente.

Gestionnaire de position	Description
pack	Le conteneur est coupé en deux zones. L'une de ces zones sera la zone occupée par le widget et l'autre la zone libre. La zone libre sera de nouveau coupée

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

	en deux au positionnement du prochain widget.
place	Positionne le widget au pixel près. Ce gestionnaire est difficile à utiliser dans la pratique.
grid	Découpe le conteneur en grille et place le widget dans une cellule de cette grille.

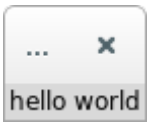
Pour placer un widget, il faut appeler les méthodes **pack(), place() ou grid()** du widget. Dans ce cours, nous verrons seulement le gestionnaire de position **grid**.

Pour reprendre notre exemple précédent, on va utiliser **grid()**, sans aucun paramètre, Tkinter se débrouillera pour placer le widget au mieux.

```
1. from tkinter import Tk, Label
2. root = Tk() # Création de la fenêtre racine
3. label = Label(root, text='Hello')
4. label.grid()
5. root.mainloop() # Lancement de la boucle principale
```



On va aller un peu plus loin en créant deux labels « hello » et « world », puis on va placer chacun d'eux dans une cellule via la méthode **grid()** en spécifiant dans quelle colonne et quelle ligne le placer via les paramètres *row* et *column*.



Voici un exemple dans lequel « hello » est affiché dans la colonne 0 et la ligne 0 et « world » dans la colonne 1 et la ligne 0.

```
1. from tkinter import Label, Tk
2. root = Tk()
3. lab1 = Label(root, text='hello')
4. lab2 = Label(root, text='world')
5. lab1.grid(column=0, row=0)
6. lab2.grid(column=1, row=0)
7. root.mainloop()
```

Si l'on souhaite afficher « hello » et « world » l'un en dessous de l'autre, on pourra faire ceci :

```
1. lab1.grid(column=0, row=0)
2. lab2.grid(column=0, row=1)
```



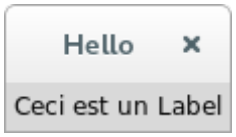
III - Les widgets disponibles dans Tkinter

Tkinter propose une dizaine de widgets, ceux-ci peuvent prendre en paramètre les options suivantes :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

Options standard	Description
foreground	La couleur du texte du widget.
activeforeground	La couleur du texte lorsque le curseur est sur le widget.
background	Couleur de fond.
activebackground	Couleur de fond lorsque le curseur est sur le widget (valable uniquement pour les widgets cliquables).
padx	Marge horizontale entre les bords du widget et son contenu.
pady	Marge verticale entre les bords du widget et son contenu.
width	Largeur du widget en taille de police.
height	Hauteur du widget en taille de police.

III-A - Le Label

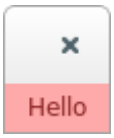


Comme vu dans l'introduction, il sert à afficher du texte, voici comment faire un label qui affiche « Ceci est un Label » :

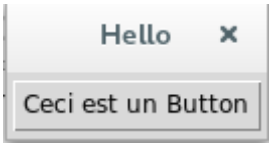
```
1. label = Label(widget_parent, text='Ceci est un Label')
```

Comme la majorité des widgets, on peut changer sa couleur en utilisant les options standards *foreground* et *background*, ou lui ajouter des marges avec *padx* et *pady*.

```
1. from tkinter import Tk, Label
2. root = Tk()
3. label = Label(root, text="Hello", foreground="#892222",
4.                background="#FFAAAA", padx="10", pady="4")
5. label.grid()
6. root.mainloop()
7. root.mainloop()
```



III-B - Les Boutons (Button)



Le widget **button** prend en paramètre un texte à afficher et une fonction à exécuter.

La fonction à exécuter ne prend pas de paramètre. Si vous souhaitez utiliser une fonction déjà écrite, mais qui prend des paramètres, vous pouvez l'encapsuler avec *partia*l comme ceci :

```
1. from functools import partial
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

```
2.
3. def foo(a, b):
4.  print(a, b)
5.
6. foo_1 = partial(foo, 3, 4) # Crée une copie de foo avec a=3 et b=4
7. foo_1() # Affiche 3, 4
8. foo_2 = partial(foo, b=3, a=4)
9. foo_2() # Affiche 4, 3
```

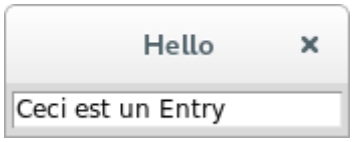
C'est ce que l'on appelle de la *curryfication*. Il existe également d'autres solutions :

- **utiliser une fermeture** ;
- **utiliser une fonction** anonyme.

L'exemple ci-dessous crée un bouton qui va exécuter la fonction **update_label()** en ayant préinitialisé les paramètres « label » et « text » grâce à *partial*. La fonction **update_label()** prend simplement un widget **Label** et remplace le texte de celui-ci par le texte fourni.

```
1. from functools import partial
2. from tkinter import *
3.
4. def update_label(label, text):
5.     """
6.     Modifie le texte d'un label.
7.     """
8.     label.config(text=texte)
9.
10. root = Tk()
11. label = Label(root, text='')
12. button = Button(root, text='clic', command=partial(update_label, label, text='Merci'))
13.
14. label.grid(column=0, row=0)
15. button.grid(column=0, row=1)
16. root.mainloop()
```

III-C - Les champs de saisie (Entry)



Ce widget sert à recueillir la saisie d'un utilisateur. Il prend en paramètre un objet de type **StringVar** qui va servir à récupérer le texte de la saisie. Si la **StringVar** est mise à jour, le champ de saisie est également modifié.

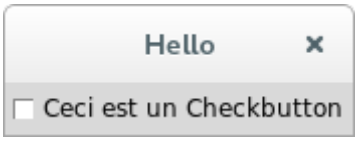
```
1. from tkinter import Tk, StringVar, Label, Entry, Button
2. from functools import partial
3.
4. def update_label(label, stringvar):
5.     """
6.     Met à jour le texte d'un label en utilisant une StringVar.
7.     """
8.     text = stringvar.get()
9.     label.config(text=text)
10.     stringvar.set('merci')
11.
12. root = Tk()
13. text = StringVar(root)
14. label = Label(root, text='')
15. entry_name = Entry(root, textvariable=text)
16. button = Button(root, text='clic',
17.                 command=partial(update_label, label, text))
18.
19. label.grid(column=0, row=0)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

```
entry_name.grid(column=0, row=1)
button.grid(column=0, row=2)

root.mainloop()
```

III-D - Les widgets checkbutton



Les widgets de type **Checkbutton** sont des cases que coche l'utilisateur. Pour connaître et modifier l'état du **Checkbutton** on utilise un objet **BooleanVar.** Cet objet est semblable à l'objet **StringVar** utilisé pour le widget **Entry**.

Tout comme les **Button**, les **Checkbutton** peuvent également prendre une fonction à exécuter via le paramètre *command*. La fonction est exécutée dès que la case est cochée ou décochée.

```
from tkinter import Tk, BooleanVar, Label, Checkbutton
from functools import partial

def update_label(label, var):
    """
    Met à jour le texte d'un label en utilisant un BooleanVar.
    """
    text = var.get()
    label.config(text=str(text))

root = Tk()
is_checked = BooleanVar(root, '1')
label = Label(root, text=str(is_checked.get()))
checkbox = Checkbutton(root, variable=is_checked,
                       command=partial(update_label, label,
                                       is_checked))

label.grid(row=0, column=0)
checkbox.grid(row=1, column=0)
root.mainloop()
```

*Vous remarquerez que le **BooleanVar** est initialisé avec la chaîne de caractères « 1 ». Si vous initialisez un **BooleanVar** avec un entier ou un booléen, appeler **get()** sur le **BooleanVar** retournera un entier.*
BooleanVar(root, True).get() → 1
BooleanVar(root, '1').get() → True

On peut également définir une autre valeur que *True* ou *False* si la case est cochée ou décochée en utilisant les options *onvalue* et *offvalue*. Par défaut leurs valeurs sont 0 et 1. Dans l'exemple ci-dessous, on va utiliser un **StringVar** à la place du **BoolVar**, en spécifiant *onvalue* à « red » et *offvalue* à « white », la **StringVar** aura la valeur « red » si la case est cochée sinon « white ».

```
from tkinter import Tk, StringVar, Label, Checkbutton
from functools import partial

def update_label(label, var):
    """
    Met à jour le texte d'un label en utilisant un BooleanVar.
    """
    text = var.get()
    label.config(text=text)

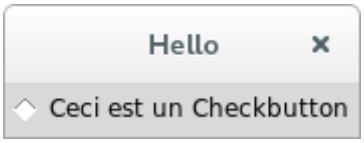

root = Tk()
is_red = StringVar(root, 'red')
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

```
14. label = Label(root, text=is_red.get())
15. checkbox = Checkbutton(root, variable=is_red, onvalue='red', offvalue='white',
 command=partial(update_label, label, is_red))
16.
17. label.grid(row=0, column=0)
18. checkbox.grid(row=1, column=0)
19. root.mainloop()
```

*Notez que l'on peut également utiliser un **IntVar** et des entiers pour onvalue et offvalue.*

III-E - Les widgets Radiobutton



Ils sont utilisés pour proposer plusieurs choix parmi lesquels l'utilisateur en choisira un seul.

Comme les **Checkbutton**, ils ont un paramètre *command* et un paramètre variable, mais il faudra utiliser le paramètre *value* pour spécifier la valeur à mettre dans la variable.

Dans l'exemple ci-dessous, on va créer trois **Radiobutton** à partir d'une liste de couleurs, en fonction du bouton radio coché, la couleur du texte d'un label change.

```
1. from tkinter import Tk, StringVar, Label, Radiobutton
2. from functools import partial
3.
4. def update_label(label, var):
5.     """
6.     Met à jour le texte de label en fonction de var.
7.     """
8.     text = var.get()
9.     label.config(text='color :' + text)
10.
11.
12. root = Tk()
13. choice = ['red', 'green', 'blue']
14. color = StringVar(root, 'red')
15. label_color = Label(root, text='color :' + color.get())
16.
17. for i, value in enumerate(choice, 1):
18.     label = Label(root, text=value)
19.     radiobutton = Radiobutton(root, variable=color, value=value,
20.                               command=partial(update_label,
21.                                               label_color,
22.                                               color))
23.
24.     label.grid(row=i, column=0)
25.     radiobutton.grid(row=i, column=1)
26.
27.
28. label_color.grid(row=0, column=0)
29. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

III-F - La Spinbox



C'est un champ avec deux petits boutons pour faire défiler les différentes valeurs proposées.

Elle s'utilise avec des nombres. On définit la plus petite valeur avec le paramètre *from_* et la plus grande valeur avec le paramètre *to*. On peut également spécifier le pas avec lequel la valeur sera incrémentée ou décrémentée. Comme ce widget hérite d'**Entry**, vous pouvez lui spécifier une variable via le paramètre *textvariable*. Comme les valeurs sont des nombres flottants, il faut utiliser une **DoubleVar**.

La **Spinbox** peut également prendre une fonction à exécuter à chaque clic sur les boutons via le paramètre *command*.

⚠ *Lorsqu'on utilise la méthode **cget()** avec le widget **Spinbox** pour récupérer la valeur du paramètre from_, il faut faire un **cget('from')** et non **cget('from_')**.*

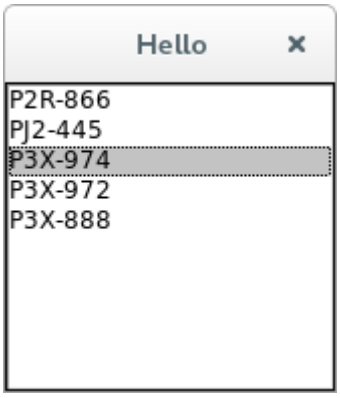
```
1. from tkinter import Tk, DoubleVar, Label, Spinbox
2. from functools import partial
3.
4. def update_label(spinbox, label, var):
5.     """
6.     Écrit 'min' ou 'max' dans label en fonction de la valeur
7.     du textvariable de spinbox
8.     """
9.     value = var.get()
10.     if value == spinbox.cget('from'):
11.         label.config(text='Min')
12.     elif value == spinbox.cget('to'):
13.         label.config(text='Max')
14.
15. root = Tk()
16. value = DoubleVar(root)
17. label = Label(text=4)
18. spinbox = Spinbox(root, textvariable=value, from_=4, to=8, increment=0.5)
19. spinbox.config(command=partial(update_label, spinbox, label, value))
20.
21. spinbox.grid(row=0, column=0)
22. label.grid(row=1, column=0)
23.
24. root.mainloop()
```

La **Spinbox** ne s'utilise pas obligatoirement avec des nombres. Dans ce cas-là, il faut utiliser le paramètre *values* à la place des paramètres *from_ to* et *increment*.

```
1. root = Tk()
2. spinbox = Spinbox(root, values=['A', 'B', 'C'])
3. spinbox.grid()
4. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

III-G - Le widget Listbox



Une liste dans laquelle l'utilisateur peut faire un ou plusieurs choix.

Les différentes manières de choisir les valeurs dans la **Listbox** sont : *single, browse, multiple,* ou *extended*.

Pour remplir la liste avec des valeurs, il faut utiliser la méthode **insert()**. Cette méthode prend en premier paramètre l'endroit où l'on souhaite insérer l'élément, puis le ou les éléments à insérer. Une autre façon de faire est d'instancier la **Listbox** avec un objet variable qui contient un *tuple*.

Option de sélection	Description
single	Un clic.
browse	Possibilité de déplacer la souris avec le clic enfoncé.
multiple	Plusieurs éléments sélectionnables et désélectionnables par un clic.
extended	*browse* avec la possibilité de prendre plusieurs éléments.

Pour savoir quelles valeurs sont sélectionnées dans la **Listbox**, il faut appeler la méthode **curselection()** qui retourne la liste des positions des éléments sélectionnés.

L'exemple ci-dessous affiche les éléments sélectionnés dans la **Listbox** lorsqu'on clique sur le bouton.

```
1. from tkinter import *
2. from functools import partial
3.
4. def show_selection(label, choices, listbox):
5.     choices = choices.get()
6.     text = ""
7.     for index in listbox.curselection():
8.         text += choices[index] + " "
9.
10.     label.config(text=text)
11.
12. root = Tk()
13. choices = Variable(root, ('P2R-866', 'PJ2-445', 'P3X-974'))
14. listbox = Listbox(root, listvariable=choices, selectmode="multiple")
15. listbox.insert('end', 'P3X-972', 'P3X-888')
16. label = Label(root, text='')
17. button = Button(root, text='Ok', command=partial(show_selection, label, choices, listbox))
18.
19. listbox.grid(row=0, column=0)
20. button.grid(row=1, column=0)
21. label.grid(row=2, column=0)
22. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

Vous avez peut-être remarqué que, dans la fonction **show_selection()**, on aurait pu récupérer la variable *choices* à partir de la **Listbox** plutôt que de la faire passer en paramètre. Il faut cependant savoir que récupérer une variable à partir d'un widget retourne le nom de la variable et non la variable elle-même. Tkinter permet cependant de récupérer le contenu d'une variable à partir de son nom avec la méthode **getvar()**.

```
1. Valeur = ma_listbox.getvar(ma_listbox.cget('listvariable'))
```

On peut également récupérer un objet variable de la façon suivante :

```
1. Variable = Variable(name=ma_listbox.cget('listvariable'))
```

III-H - Le widget Scale



Ce widget est une barre le long de laquelle on déplace un curseur pour sélectionner une valeur. Le besoin auquel répond le **Scale** est semblable à une **Spinbox**, mais l'aspect visuel est différent. Comme pour la **Spinbox**, vous pouvez indiquer la valeur *min* et *max* avec les paramètres *from_* et *to*.

Le paramètre *showvalue* permet d'afficher ou non la valeur sélectionnée par le curseur.

Vous pouvez également afficher des marques à intervalle régulier grâce au paramètre *tickinterval.* L'orientation de la barre est définie par le paramètre *orient* qui peut prendre pour valeur *h* pour horizontal ou *v* pour vertical. Un label peut également être défini.

Comme beaucoup d'autres widgets, le **Scale** peut prendre une fonction à exécuter à chaque changement de valeur via le paramètre *command.*

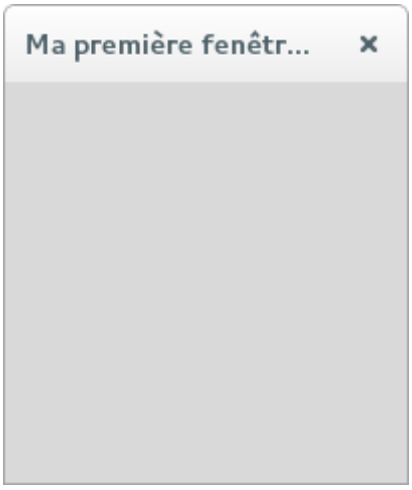
```
1. from tkinter import IntVar, Tk, Scale, Entry
2. 
3. root = Tk()
4. value = IntVar(root)
5. scale = Scale(root, from_=4, to=16, showvalue=True, label='degres',
6. variable=value, tickinterval=4, orient='h')
7. entry = Entry(root, textvariable=value)
8. 
9. scale.grid(row=0, column=0)
10. entry.grid(row=0, column=1)
11. 
12. root.mainloop()
```

IV - Les conteneurs

Les conteneurs sont des widgets destinés à contenir d'autres widgets. Dans un premier temps, nous allons voir plus en détail un widget que l'on a déjà utilisé, la fenêtre.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

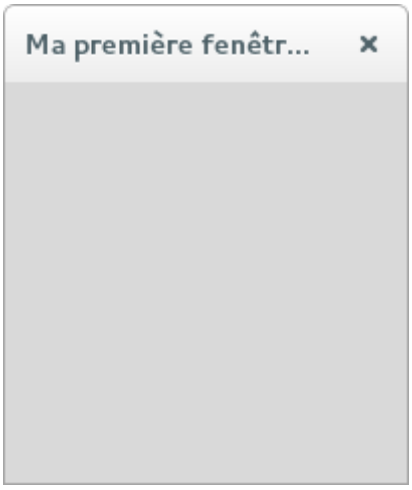
IV-A - Les fenêtres (Toplevel)



Les fenêtres, et plus particulièrement la fenêtre racine, sont la base d'une application Tkinter.
Elles ont plusieurs méthodes qui sont héritées de **wm** (Windows Manager) et qui permettent de les personnaliser.

Méthode	Description
title	Cette méthode change le titre de la fenêtre.
resizable	Autorise le redimensionnement de la fenêtre horizontalement et verticalement grâce à deux booléens passés en paramètre.

```
1. from tkinter import Tk
2. root = Tk() # Création de la fenêtre racine
3. root.title('Ma première fenêtre tkinter') # Ajout d'un titre
4. root.resizable(True, False) # autoriser le redimensionnement vertical.
5. root.mainloop() # Lancement de la boucle principale
```



La méthode **geometry()** permet d'indiquer la taille et le positionnement de la fenêtre. Elle prend en paramètre une chaîne de caractères au format :

{taille_horizontale}x{taille_verticale}

qui peut être suivie de :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

+{position_a_partir_du_bord_gauche}+{position_a_partir_du_haut}

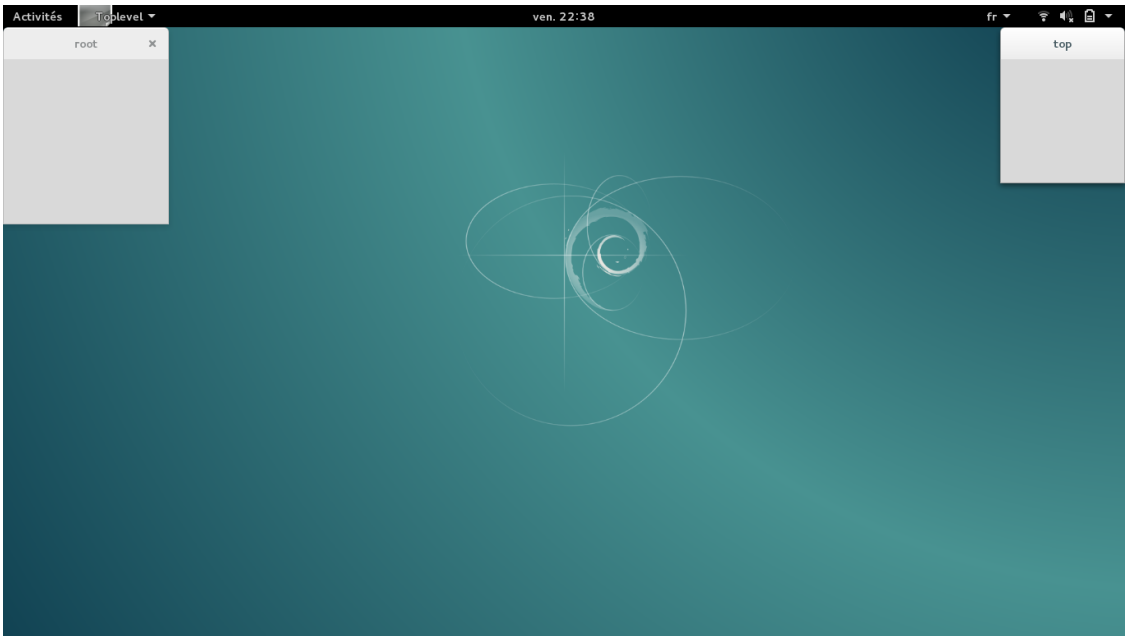
En remplaçant les « + » par des « - » on indique la positon à partir du bord droit et du bas.

Exemple d'une fenêtre longue de 400 px et haute de 300 px, placée à 10 px du bord droit et 100 px du haut.

```
1. from tkinter import Tk
2. root = Tk() # Création de la fenêtre racine
3. root.geometry("400x300-10+100")
4. root.mainloop() # Lancement de la boucle principale
```

Si l'on veut créer une deuxième fenêtre, il faut le faire en utilisant le widget **TopLevel**. Cette deuxième fenêtre peut être transitoire à la fenêtre parent. Cela signifie qu'elle restera toujours devant la fenêtre parent, même si la fenêtre parent reprend le focus. De plus une réduction de la fenêtre parent réduit également la fenêtre transitoire.

```
1. from tkinter import Tk
2. root = Tk()
3. root.title('root')
4. root.geometry('200x200+0+0')
5. top = Toplevel(root)
6. top.title('top')
7. top.geometry('150x150-0+0')
8. top.transient(root)
9. root.mainloop()
```



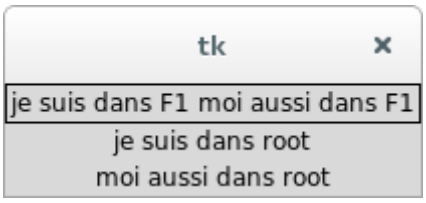
IV-B - le widget Frame

Une **Frame** va servir à regrouper plusieurs widgets entre eux. Cela peut simplifier leur placement par la suite.

```
1. from tkinter import Tk, Label, Frame
2.
3. root = Tk()
4. f1 = Frame(root, bd=1, relief='solid')
5. Label(f1, text='je suis dans F1').grid(row=0, column=0)
6. Label(f1, text='moi aussi dans F1').grid(row=0, column=1)
7.
8. f1.grid(row=0, column=0)
9. Label(root, text='je suis dans root').grid(row=1, column=0)
10. Label(root, text='moi aussi dans root').grid(row=2, column=0)
11.
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

```
12. root.mainloop()
```



IV-C - LabelFrame

LabelFrame est une *frame* avec un label en titre.

On peut choisir la position du titre avec le paramètre *labelanchor* qui prend en paramètres les points cardinaux : nw, n, ne, e, se, s, sw, w.

IV-D - le widget PanedWindow



Le **PanedWindow** et un conteneur qui permet de partager le même espace visuel entre plusieurs conteneurs appelés *pane*. On viendra ensuite ajouter des widgets aux *pane* avec la méthode **add()**. Le **PanedWindow** peut prendre plusieurs options : *showhandle* affiche une poignée s'il est à *True*, la taille de la poignée peut être définie via l'option *handlesize* et *handlepad* permet de définir à quelle distance du bord mettre la poignée.

L'option *sashwidth* indique la taille du séparateur et *sashrelief* permet de définir son style de relief. Cette option peut prendre les valeurs *flat, groove, raised, ridge, solid,* ou *sunken*. On peut également définir l'espace entre le séparateur et le contenu via l'option *sashpad*.

```
1. from tkinter import Tk, PanedWindow, Label
2. root = Tk()
3. root.title('Hello')
4. paned = PanedWindow(root, handlesize=4, showhandle=True, sashrelief='sunken')
5. l1 = Label(paned, text='gauche', height=100, background="white")
6. l2 = Label(paned, text='droite', height=100, background="white")
7. paned.add(l1, height=100, sticky="ew")
8. paned.add(l2, height=100, sticky="ew")
9. paned.grid(sticky="ew", row=1, column=1)
10. root.grid_columnconfigure(1, weight=1)
11. root.mainloop()
```

V - Un peu plus loin avec grid()

On a juste vu les deux paramètres de base de **grid()** qui sont *column* et *row*, mais **grid()** est beaucoup plus souple que ça ; on peut par exemple demander à une cellule de s'étendre horizontalement avec *columnspan* ou verticalement avec *rowspan*.

```
1. root = Tk()
2. Label(root, text='l1', bg='white').grid(row=0, column=0, columnspan=2)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

```
3. Label(root, text='l2', bg='red').grid(row=1, column=0)
4. Label(root, text='l3', bg='green').grid(row=1, column=1)
5. root.mainloop()
```

Dans l'exemple ci-dessus, l1 s'étend de l2 à l3

À partir d'un widget, on peut savoir de quelle façon il a été mis dans la grille avec la fonction **grid_info()** qui retourne un dictionnaire avec les paramètres utilisés lors de l'appel à la méthode **grid()**.

Centrer le widget dans une cellule ou l'étendre avec l'option *sticky* : Il peut arriver qu'un widget ne prenne pas toute la place de la cellule, dans ce cas-là vous pouvez indiquer où il doit se placer en donnant une combinaison de points cardinaux avec *sticky.*

Cet exemple va vous permettre de tester comment agit le paramètre *sticky* sur le widget :

```
1. from tkinter import *
2. from functools import partial
3.
4. # Dessiner une mosaïque de 3 * 3 labels.
5. # Tous les labels sont blancs sauf celui au centre qui est rouge.
6. # Le label rouge à une taille de 1 sur 1 mais les labels sur ses
7. # côtés ont une taille de 3 ce qui fait que le label rouge ne prend
8. # pas toute la place de sa cellule.
9. root = Tk()
10. Label(root, height=1, width=1, bg='white').grid(row=0, column=0)
11. Label(root, height=1, width=3, bg='white').grid(row=0, column=1)
12. Label(root, height=1, width=1, bg='white').grid(row=0, column=2)
13. Label(root, height=3, width=1, bg='white').grid(row=1, column=0)
14. label = Label(root, height=1, width=1, bg='red')
15. label.grid(row=1, column=1)
16. Label(root, height=3, width=1, bg='white').grid(row=1, column=2)
17. Label(root, height=1, width=1, bg='white').grid(row=2, column=0)
18. Label(root, height=1, width=3, bg='white').grid(row=2, column=1)
19. Label(root, height=1, width=1, bg='white').grid(row=2, column=2)
20.
21. def update_sticky_option(label, coord, is_checked):
22.     '''
23.     Modifier le paramètre sticky d'un widget.
24.     '''
25.     sticky = label.grid_info()['sticky']
26.     if is_checked.get():
27.         sticky += coord
28.     else:
29.         sticky = sticky.replace(coord, '')
30.     label.grid_configure(sticky=sticky)
31.
32. # Créer 4 Checkbox pour pousser les valeurs 'n', 's', 'e', et 'w'.
33. # au paramètre sticky du label.
34. for i, coord in enumerate('nsew', 3):
35.     is_checked = BooleanVar(root)
36.
37.     Label(root, text=coord).grid(row=i, column=0)
38.     Checkbutton(root, variable=is_checked,news
39.                 command=partial(update_sticky_option,
40.                                 label, coord,
41.                                 is_checked)).grid(row=i, column=1)
42.
43. root.mainloop()
```

On peut également ajouter des marges horizontales avec l'option *padx* et verticales avec *pady.*

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

VI - Les événements

Il est possible de récupérer des événements, comme la frappe d'une touche ou un clic de souris pour effectuer un traitement spécial. Pour qu'un widget puisse traiter les événements, il faut que celui-ci ait le focus. Généralement, un widget prend le focus lorsque l'on clique dessus. Les événements peuvent ensuite être traités par une fonction.

Pour définir un type d'événement auquel on souhaite réagir, on utilise une chaîne de caractères qui commencera par le caractère « < » et finira par le caractère « > ». Entre ces signes, on indiquera le type d'événement auquel on veut réagir :

Type d'événement	Description
ButtonPress	Pression d'un bouton de la souris.
ButtonRelease	Relâchement d'un bouton de la souris.
KeyPress	Pression sur une touche du clavier.
KeyRelease	Relâchement d'une touche du clavier.

exemple :

Événement	Description
<KeyPress>	Pour l'enfoncement d'une touche.

Pour ajouter plus de précision, on peut ajouter un détail en le séparant du type d'événement par un tiret comme ceci <type-détail>. Le détail peut être un bouton de souris en particulier. Dans ce cas on met un chiffre de 1 à 5 ou une touche particulière.

Événement	Description
<ButtonPress-1>	Clic gauche.
<KeyPress-a>	Pression sur la touche 'a'.

On peut également préfixer le type d'événement avec un modificateur, ce qui va permettre d'avoir des choses complexes comme un clic avec la touche *alt* enfoncée.

Les modificateurs utilisables sont :

Modificateur	Description
Control	La touche Control sera pressée.
Shift	La touche Shift sera pressée.
Alt	La touche Alt sera pressée.

Les différents boutons de la souris B1, B2, B3, B4 et B5 :

Modificateur	Description
Double	Double clic.
Triple	Triple clic.
Quadruple	Quadruple clic.

exemple :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

Événement	Description
<Control-Alt-Double-ButtonPress-1>	Faire un double clic pendant que les touche Alt + Ctrl sont enfoncées
<B1-KeyPress-Alt_L>	Appuyer sur la touche Alt de gauche pendant que le bouton de gauche de la souris est pressé.

Remarquez que l'on utilise *Alt_L* pour indiquer le détail et *Alt* pour indiquer le modificateur.

Pour lier une fonction à un événement, on utilise la méthode **bind()**. La fonction qui est liée prend en paramètre un objet de type **Event** qui contient de nombreux attributs comme le moment de l'événement, sa positon s'il s'agit d'un clic… Pour avoir une meilleure idée de ce que contient l'objet **Event**, je vous invite à exécuter le code ci-dessous qui affiche le contenu de l'objet **Event** en fonction des clics ou de l'appui d'une des touches du clavier.

```
1. from tkinter import *
2. from pprint import pformat
3. from functools import partial
4.
5. def print_event(label, event):
6.     label.config(text=pformat(vars(event)))
7.
8. root = Tk()
9. frame = Frame(root, bg='white', height=100, width=400)
10. entry = Entry(root)
11. label = Label(root)
12.
13. frame.grid(row=0, column=0)
14. entry.grid(row=1, column=0, sticky='ew')
15. label.grid(row=2, column=0)
16.
17. frame.bind('<ButtonPress>', partial(print_event, label))
18. entry.bind('<KeyPress>', partial(print_event, label))
19. root.mainloop()
```

Le second exemple affiche trois champs de texte. Lorsque l'on appuie sur la touche *Entrée*, le focus passe sur le champ suivant.

```
1. from tkinter import *
2.
3. def change_focus(event):
4.     current_row = event.widget.grid_info()['row']
5.     parent = event.widget.master
6.     next_widgets = parent.grid_slaves(column=0, row=current_row + 1)
7.     if next_widgets:
8.         next_widgets[0].focus()
9.
10. root = Tk()
11. entry_group = Frame(root)
12. for i in range(3):
13.     entry = Entry(entry_group)
14.     entry.grid(row=i, column=0)
15.     entry.bind('<Key-Return>', change_focus)
16.     entry_group.grid(row=0, column=0)
17.
18. root.mainloop()
```

Lorsque l'on souhaite ajouter plusieurs fonctions à un même widget, il faut mettre le troisième paramètre de **bind()** à *True* sinon toutes les fonctions liées seront remplacées.

Lorsque l'événement se produit, les fonctions sont appelées les unes après les autres sauf si l'une des fonctions retourne la chaîne *break*. À ce moment-là, aucune fonction liée à l'événement ne sera appelée.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

Dans l'exemple ci-dessous, on lie d'abord f1, puis f2. Si l'on presse la touche 'a', f1 retourne *break* et f2 n'est pas appelée. On remarque également que la fonction du widget qui traite l'événement par défaut n'est pas appelée.

```
1. def f1(label, event):
2.     label.config(text=event.keysym)
3.     if event.keysym == 'a':
4.         return 'break'
5.
6. def f2(label, event):
7.     label.config(text=event.keycode)
8.
9. root = Tk()
10. entry = Entry(root)
11. label_1 = Label(root)
12. label_2 = Label(root)
13.
14. entry.grid(row=0, column=0)
15. label_1.grid(row=1, column=0)
16. label_2.grid(row=2, column=0)
17. entry.bind('<Key>', partial(f1, label_1))
18. entry.bind('<Key>', partial(f2,
 label_2), True) # Sans le True, f2 remplace les autres fonctions liées
19. root.mainloop()
```

VII - Organiser son code

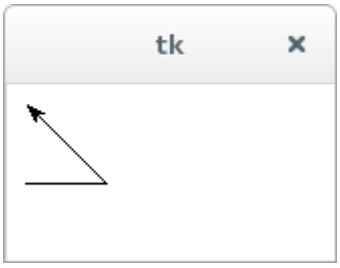
L'idée, pour organiser son code, est de créer ses propres widgets en héritant de la classe **Frame** et d'encapsuler les différents widgets et la logique d'interaction des widgets dans cette classe.

Dans l'exemple ci-dessous, on va créer un widget **VotingBox** qui permettra d'afficher une question dans un label et différentes réponses possibles dans des boutons.

```
1. class VotingBox(Frame):
2.     def __init__(self, *args, question='', options=None, **kwargs):
3.         Frame.__init__(self, *args, **kwargs)
4.         Label(self, text=question).grid(column=0, row=0,
5.                                         columnspan=len(options))
6.         self.int_vars = {}
7.         for num_col, option in enumerate(options):
8.             self.int_vars[option] = IntVar(self)
9.             Button(self, text=option,
10.                    command=partial(self._update_int_var,
11.                                    option)).grid(column=num_col,
12.                                                  row=1)
13.             label = Label(self, textvariable=self.int_vars[option])
14.             label.grid(column=num_col, row=2)
15.     def _update_int_var(self, option):
16.         int_var = self.int_vars[option]
17.         int_var.set(int_var.get() + 1)
18. root = Tk()
19. VotingBox(root,
20.           question='Que pensez-vous de tkinter ?',
21.           options=[':-)', ':-|', ':-(']).grid()
22. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

VII-A - Le widget Canvas



Bien que le **Canvas** soit un widget comme les autres, sa complexité et son nombre de méthodes font que nous le voyons dans un chapitre séparé. Le widget **Canvas** permet de dessiner des formes telles que des lignes, des cercles…, et de les manipuler par la suite. Pour dessiner des formes, on utilise un ensemble de méthodes dont les noms sont préfixés par « create_ » tel que **create_rectangle()** ou **create_line()**. Ces méthodes retournent l'*id* de la forme créée.

Lorsque l'on va créer une forme, il faut lui transmettre des coordonnées pour pouvoir positionner celle-ci et la dimensionner.

Les coordonnées sont passées en paramètres de la façon suivante :

```
1. create_line(x1, y1, x2, y2, ...)
```

Les coordonnées des points peuvent être regroupées dans un *tuple*, Tkinter s'en accommodera. Ainsi, les trois lignes ci-dessous sont équivalentes :

```
1.      create_line((10,10), (50,50), (10, 50))
2.      create_line((10,10, 50,50, 10, 50))
3.      create_line(10,10, 50,50, 10, 50)
```

VII-A-1 - Créer des lignes

Pour dessiner une ligne sur le **Canvas**, on utilise la méthode **create_line()**, puis on passe en paramètres une série de points qui vont être reliés par la ligne. Sur un **Canvas**, le point en haut à droite a pour coordonnées (0, 0).

```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_line((10,10), (50,50), (10, 50))
5. canvas.grid()
6. root.mainloop()
```

Lorsque l'on trace une ligne, on peut définir la couleur et la taille de celle-ci avec respectivement les options *fill* et *width*.

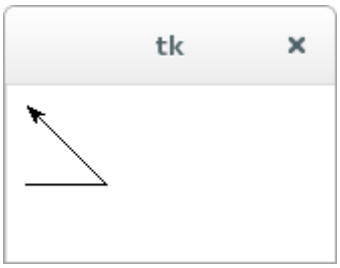
```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_line((10,10), (50,50), (10, 50), fill='red', width=2)
5. canvas.grid()
6. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/



On peut également mettre des flèches au bout de notre ligne avec l'option *arrow.* Cette option peut prendre trois valeurs : *first, last* ou *both* pour dessiner respectivement la flèche au début, à la fin ou aux deux extrémités de la ligne.

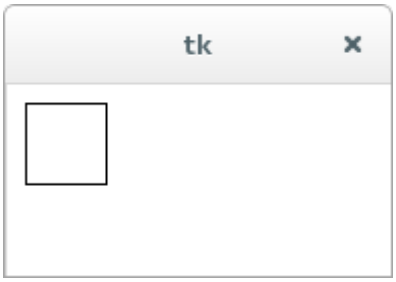
```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_line((10,10), (50,50), (10, 50), arrow="first")
5. canvas.grid()
6. root.mainloop()
```



VII-A-2 - Créer des rectangles

la méthode **create_rectangle()** prend en paramètres deux couples de coordonnées :

```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_rectangle((10,10), (50,50))
5. canvas.grid()
6. root.mainloop()
```



Par défaut, le carré n'a pas de fond, mais on peut lui en donner un en passant une couleur à l'option *fill.*

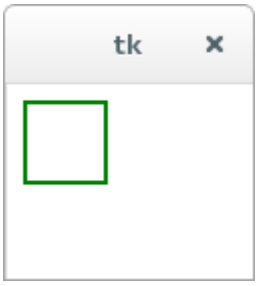
```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_rectangle((10,10), (50,50), fill='red')
5. canvas.grid()
6. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/



La taille de la bordure peut être modifiée avec l'option *width* et la couleur avec l'option *outline :*

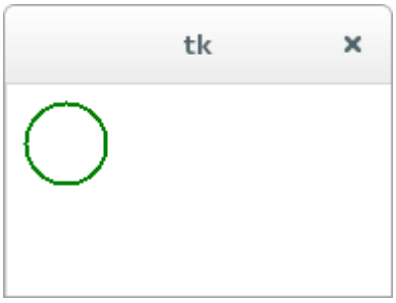
```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_rectangle((10,10), (50,50), width=2, outline='green')
5. canvas.grid()
6. root.mainloop()
```



VII-A-3 - Créer des ovales

La méthode **create_oval()** permet de dessiner un ovale. Elle prend en paramètres deux points et dessine un ovale dont le périmètre est englobé dans les coordonnées données. À l'instar du rectangle, on peut définir les paramètres *fill, width* et *outline*.

```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_oval((10,10), (50,50), width=2, outline='green')
5. canvas.grid()
6. root.mainloop()
```



VII-A-4 - Créer des arcs

La méthode **create_arc()** permet de tracer un arc délimité entre deux angles. Elle prend en paramètres deux points représentant un rectangle dans lequel l'arc sera englobé.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

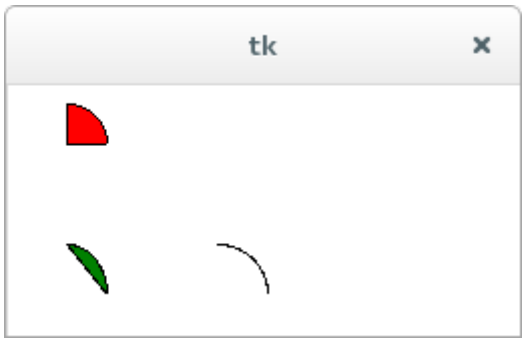
Le paramètre *extent* permet de définir l'angle de l'arc en degrés. Le paramètre *start* permet de choisir où commence l'arc. La valeur est à donner en degrés, de 0 à 360 avec la signification suivante :

- 0 ou 360 degrés est positionné comme l'aiguille d'une montre à 3 heures ;
- 90 degrés est positionné comme l'aiguille d'une montre à midi ;
- 180 est positionné comme l'aiguille d'une montre à 9 heures ;
- 270 est positionné comme l'aiguille d'une montre à minuit.

Il y a également trois styles que l'on peut appliquer sur les arcs :

- *arc* - seul le périmètre de l'arc est tracé ;
- *pie slice* - idéale pour dessiner les graphiques en camembert ;
- *chord* - le même arc que celui de Robin des bois, une ligne relie les deux extrémités de l'arc entre elles.

```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_arc((10,10), (50,50), fill='red',
5.                   start=0, extent=90, style='pieslice')
6. canvas.create_arc((80,80), (130,130),
7.                   start=0, extent=90, style='arc')
8. canvas.create_arc((10,80), (50,130), fill='green',
9.                   start=0, extent=90, style='chord')
10. canvas.grid()
11. root.mainloop()
```

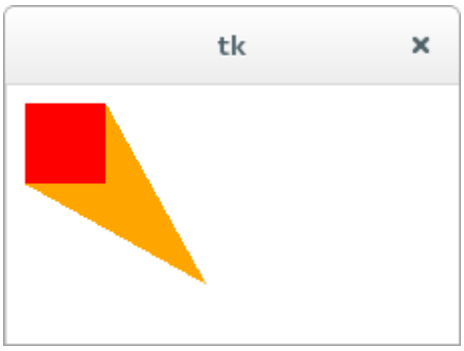


VII-A-5 - Créer des polygones

On donne une série de points et Tkinter tracera un polygone en passant par tous les points donnés. La méthode **create_polygone()** prend les mêmes arguments que la méthode **create_rectangle()**.

```
1. from tkinter import Tk, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. canvas.create_polygon((10,10), (10,50), (50,50), (50,10), fill='red')
5. canvas.create_polygon((10,50), (50,50), (50,10), (100, 100), fill='orange')
6. canvas.grid()
7. root.mainloop()
```

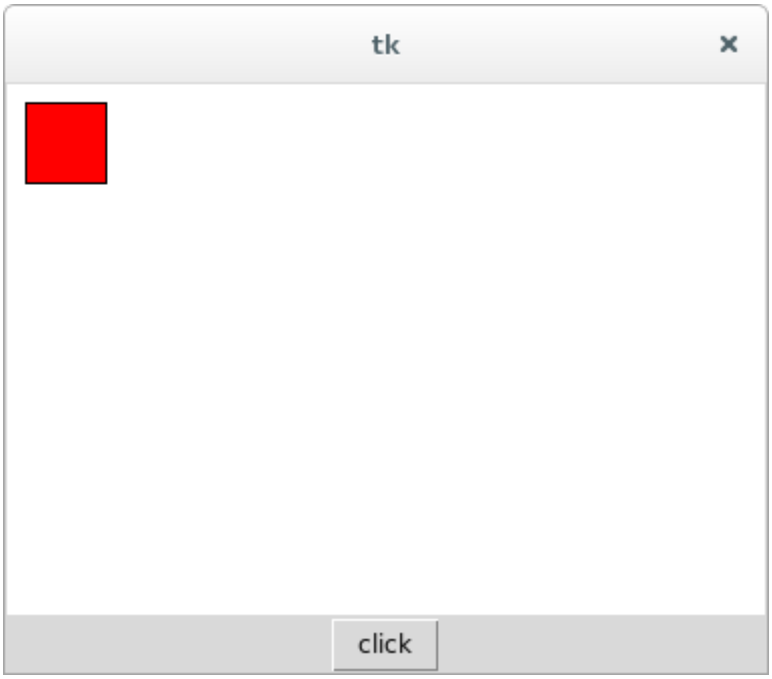

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/



VII-A-6 - Interagir avec les objets du Canvas

Lorsque vous utilisez les méthodes de type **create_***, le **Canvas** va retourner l'identifiant de l'objet créé. On va ensuite pouvoir utiliser cet identifiant pour pouvoir modifier l'objet avec **itemconfig()** ou lire ses propriétés avec **itemcget()**

```
1. from tkinter import Tk, Canvas
2. from functools import partial
3. def update_item(canvas, item_id):
4.     """
5.     Change la couleur de l'item à rouge s'il
6.     est bleu et vice versa.
7.     """
8.     fill = canvas.itemcget(item_id, 'fill')
9.     if fill == 'red':
10.         canvas.itemconfig(item_id, fill='blue')
11.     else:
12.         canvas.itemconfig(item_id, fill='red')
13. root = Tk()
14. canvas = Canvas(root, background="white")
15. rectangle_id = canvas.create_rectangle((10,10), (50,50), fill='red')
16. button = Button(root, text="click", command=partial(update_item,
17.                           canvas, rectangle_id))
18. canvas.grid()
19. button.grid()
20. root.mainloop()
```



Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

VII-A-7 - Avoir des événements sur les objets du Canvas

On peut demander à un objet du **Canvas** d'écouter des événements via la méthode **bind()**. Celle-ci s'utilise comme la méthode **bind()** des widgets.

Les événements écoutables sont *Enter, Leave, ButtonPress, Motion* et *KeyPress*.

La fonction qui sera exécutée va recevoir en premier paramètre un objet **event**

qui contient le **Canvas** dans l'attribut widget.

Pour savoir quel objet du **Canvas** est concerné par l'événement, il y a deux solutions :

faire en sorte que l'*id* de l'objet soit passé en paramètre de la fonction utilisée dans **bind()** ;

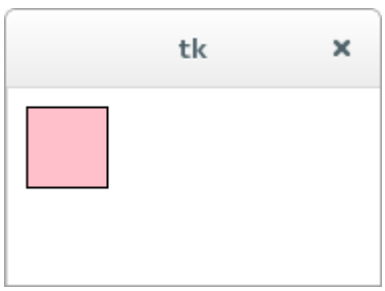
```
1. def on_enter(item_id, event):
2.     event.widget.itemconfig(item_id, fill="red")
3. canvas.tag_bind(item_id, '<Enter>', partial(on_enter, item_id))
```

ou utiliser la méthode **find_closest()** du **Canvas** qui retourne la liste des items les plus proches d'une position donnée.

```
1. def on_enter(event):
2.     for item_id in canvas.find_closest(event.x, event.y)
3.         event.widget.itemconfig(item_id, fill="red")
4. canvas.tag_bind(item_id, '<Enter>', on_enter)
```

Dans l'exemple ci-dessous, le rectangle se colorie en orange quand le pointeur de la souris entre dedans, et en rose quand celui-ci en sort :

```
1. from tkinter import Tk, Canvas, Label
2. from functools import partial
3. ENTER = '7'
4. LEAVE = '8'
5. def change_color_on_enter_or_leave(event):
6.     canvas = event.widget
7.     item_ids = canvas.find_closest(event.x, event.y)
8.     for item_id in item_ids:
9.         if event.type == ENTER:
10.             canvas.itemconfig(item_id, fill="orange")
11.         elif event.type == LEAVE:
12.             canvas.itemconfig(item_id, fill="pink")
13. root = Tk()
14. canvas = Canvas(root, background="white")
15. label = Label(root)
16. rectangle_id = canvas.create_rectangle((10,10), (50,50), fill='red')
17. canvas.tag_bind(rectangle_id, '<Enter>', change_color_on_enter_or_leave)
18. canvas.tag_bind(rectangle_id, '<Leave>', change_color_on_enter_or_leave)
19. canvas.grid()
20. label.grid()
21. root.mainloop()
```



Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/

VII-A-8 - Appliquer des tags sur les objets du Canvas

On peut donner des étiquettes, appelées tags, aux objets du **Canvas**. Cela va permettre de manipuler des groupes d'objets. On peut néanmoins utiliser la fonction **find_withtag()** pour récupérer tous les tags d'un item donné.

```
1. from tkinter import *
2. from functools import partial
3. ENTER = '7'
4. LEAVE = '8'
5. def change_color_on_enter_or_leave(item_id, event):
6.     canvas = event.widget
7.     if event.type == ENTER:
8.         canvas.itemconfig(item_id, fill="orange")
9.     elif event.type == LEAVE:
10.         canvas.itemconfig(item_id, fill="red")
11. root = Tk()
12. canvas = Canvas(root, background="white")
13. label = Label(root)
14. for x in range(0, 200, 50):
15.     for y in range(0, 200, 50):
16.         item_id = canvas.create_rectangle((x, y), (x + 40, y + 40),
17.                                                 fill='red', tag='rectangle')
18. for item_id in canvas.find_withtag('rectangle'):
19.     canvas.tag_bind(item_id, '<Enter>',
20.                             partial(change_color_on_enter_or_leave, item_id))
21.     canvas.tag_bind(item_id, '<Leave>',
22.                             partial(change_color_on_enter_or_leave, item_id))
23. canvas.grid()
24. label.grid()
25. root.mainloop()
```

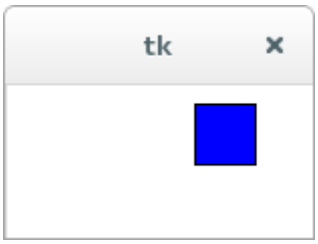
VII-B - Les animations

Pour faire des animations, on va utiliser la méthode **after()** qui va demander à la boucle d'événements de Tkinter d'exécuter une fonction au bout d'un certain temps.

Dans l'exemple qui suit, on va dessiner un carré. Puis on va appeler la fonction **move_rectangle()** qui va déplacer notre carré de 4 px sur la droite en changeant ses coordonnées. Après avoir fait cela, notre fonction va demander à Tkinter de la réexécuter au bout de 40 ms.

```
1. from tkinter import Canvas, Tk
2. root = Tk()
3. canvas = Canvas(root, background="white")
4. recta_id = canvas.create_rectangle((10,10),(40, 40), fill="blue")
5. canvas.grid()
6. def move_rectangle():
7.     cobords = canvas.coords(rect_id)
8.     if coords[0] >= 300:
9.         coords[0] = 10
10.         coords[2] = 40
11.     else:
12.         coords[0] += 4
13.         coords[2] += 4
14.     root.after(40, move_rectangle)
15.     canvas.coords(rect_id, *tuple(coords))
16. root.after(40, move_rectangle)
17. root.mainloop()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/



VII-C - Le widget Scrollbar

Les **Scrollbar** sont des widgets permettant de déplacer le cadre de vue d'un autre widget. Ils peuvent être utilisés avec le widget **Canvas**. On peut choisir l'orientation de la **Scrollbar** avec l'option *orient* qui prend en valeur *horizontal* ou *vertical*.

Si l'on souhaite qu'une **Scrollbar** déplace le cadre de vue horizontal d'un widget, il faut passer la méthode **xview()** du widget à l'option *command* de la **Scrollbar**. Il faudra par la suite configurer notre widget pour que son paramètre *xscrollcommand* pointe sur la méthode **set()** de la **Scrollbar**.

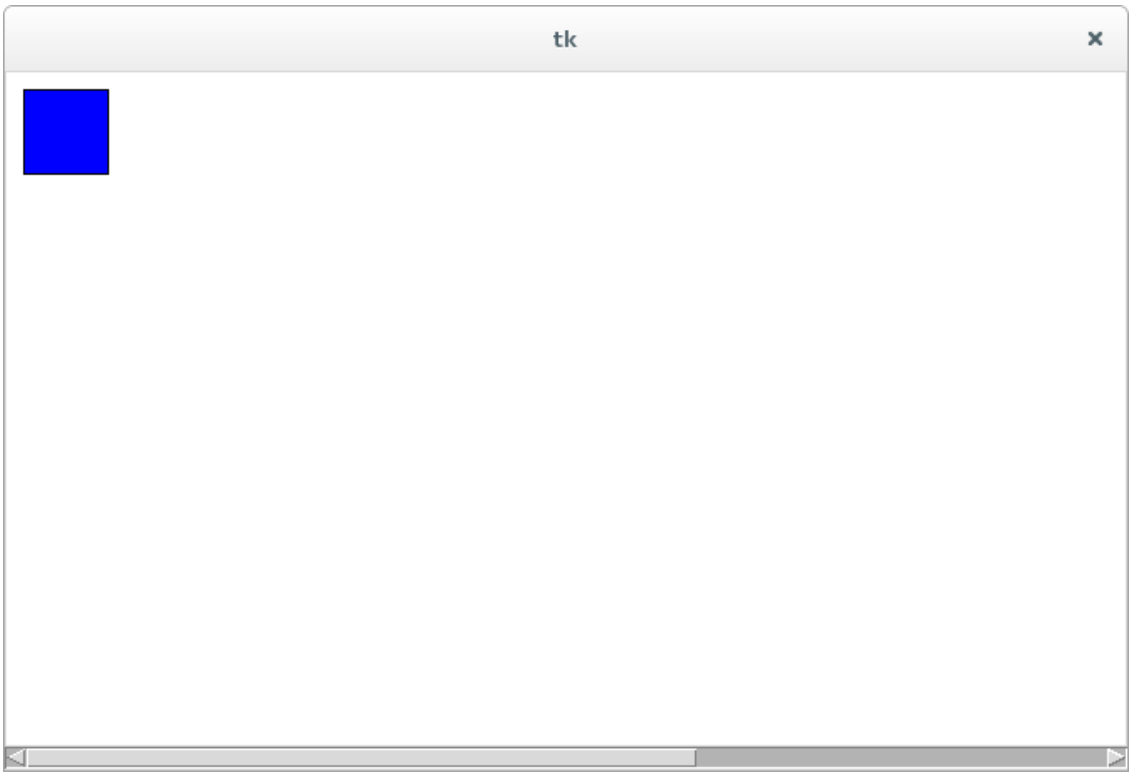
Si l'on souhaite manipuler le cadre de vue verticalement, on utilisera respectivement **yview()** et *yscrollcommand* à la place de **xview()** et *xscrollcommand.*

Dans l'exemple ci-dessous, on va créer un **Canvas** dont le cadre de vue est de 400 par 400 pixels, mais sa taille réelle est de 1070 par 400 pixels. Une **Scollbar** horizontale est utilisée pour déplacer le cadre de vue.

```
1. from tkinter import Tk, Scrollbar, Canvas
2. root = Tk()
3. canvas = Canvas(root, background="white",
4.                            scrollregion=(0, 0, 1070, 400),
5.                            width=400, height=400)
6. scroll = Scrollbar(root, orient="horizontal", command=canvas.xview)
7. canvas.config(xscrollcommand=scroll.set)
8. canvas.create_rectangle((10,10),(60,60), fill="blue")
9. canvas.create_rectangle((1010,10),(1060,60), fill="blue")
10. root.columnconfigure(1, weight=1)
11. canvas.grid(sticky="ew", column=1)
12. scroll.grid(sticky="ew", column=1)
13. root.mainloop()
```

Les **Scrollbar** peuvent être utilisées avec n'importe quel widget possédant le paramètre *xscrollcommand* ou *yscrollcommand.*

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/



VIII - Conclusion

Ce tutoriel sur Tkinter est maintenant terminé. Vous avez pu voir les principaux widgets de Tkinter, leurs mises en place ainsi que la gestion des événements. Pour ceux qui souhaitent aller plus loin, voici une documentation complète **http://infohost.nmt.edu/tcc/help/pubs/tkinter/web/index.html**.

Il peut être intéressant de consulter directement la documentation de Tcl/Tk : **http://www.vuibert.fr/sites/default/files/ressources-publiques/9782711748785-tdm2.pdf**

IX - Remerciements

- Je remercie la communauté développez.com pour les ressources mises en place, qui permettent à chacun d'apprendre et de cultiver sa curiosité. Je remercie particulièrement **deusyss**, **f-leb** et **lejimi** pour la relecture technique de cet article et **FRANOUCH** pour la relecture orthographique.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2016 Vincent Maillol. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://vincent.developpez.com/cours-tutoriels/python/tkinter/apprendre-creer-interface-graphique-tkinter-python-3/